“Campus Placement Hub: Intelligent Student Record Search and Allocation Engine using Data Structures and Algorithms”

Project Report submitted for the partial fulfilment of grade for the course
Data Structures and Algorithms - 3 (25CS2103E)
in
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
by

SUBHASH : 2520030391
AKHIL : 2520030423
REVANTH REDDY : 2520030424

Team Number: 21 | Section: C

Under the Guidance of
Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled “Campus Placement Hub: Intelligent Student Record Search and Allocation Engine using Data Structures and Algorithms” is a record of Bonafide work of Subhash – 2520030391, Akhil Kumar – 2520030423, Revanth Reddy – 2520030424 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Subhash – 2520030391
Akhil Kumar – 2520030423
Revanth Reddy – 2520030424

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled “Campus Placement Hub: Intelligent Student Record Search and Allocation Engine using Data Structures and Algorithms” is a bonafide work done and submitted by SUBHASH – 2520030391, AKHIL KUMAR – 2520030423, REVANTH REDDY – 2520030424 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, during the academic year 2026-27.

Signature of the Guide
(Dr. S. Vinay Kumar)
Associate Professor, Dept. of CSE

Signature of the HOD
(Dr. P. Venkateshwara Rao)
Associate Professor, HOD CSE

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our deepest sense of gratitude and sincere thanks to our esteemed project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his invaluable mentorship, consistent guidance, insightful encouragement, and constructive feedback throughout the design, implementation, and testing phases of this project. His deep expertise in algorithmic thinking and data structures has been profoundly instrumental in steering this project to its successful completion.

We convey our sincere thanks and heartfelt gratitude to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing excellent academic infrastructure, advanced computational laboratory resources, and a stimulating learning environment that enabled us to carry out our project smoothly and effectively.

We also take this opportunity to express our sincere appreciation to the management of Koneru Lakshmaiah Education Foundation (KLH Bowrampet Campus), all faculty members, and technical staff of the Computer Science and Engineering department for their continuous cooperation. Finally, we thank our parents, family members, and peers for their constant moral support, collaboration, and inspiration throughout this academic endeavor.

Subhash – 2520030391
Akhil Kumar – 2520030423
Revanth Reddy – 2520030424

ABSTRACT

Modern educational institutions and university campus recruitment systems manage large collections of semi-structured student profile documents containing academic details, technical competencies, projects, certifications, and career aspirations. In conventional recruitment software, queries are restricted to primary keys such as roll numbers or exact names, ignoring the rich contextual text in candidate profiles. To address this limitation, this project designs and implements the “Campus Placement Hub: Intelligent Student Record Search and Allocation Engine”, an end-to-end algorithmic platform developed in Java that combines string processing, fuzzy matching, and network-flow allocation.

The system processes a benchmark corpus of 500 student profile documents (SR001 to SR500) totalling over 3.2 MB of text. The search subsystem implements the linear-time exact matching algorithms Knuth-Morris-Pratt (KMP), using the Longest Proper Prefix which is also a Suffix (LPS) array, and the Z-Algorithm. For multi-keyword technical screening, an Aho-Corasick automaton performs simultaneous dictionary matching in O(Text Length + Matches) time after construction. A Suffix Array supports binary-search substring discovery, and Levenshtein Edit Distance provides typo tolerance and query suggestions. A Deque/HashMap Search History module maintains a most-recently-used query cache and query-frequency metrics.

For candidate-to-company allocation, the platform formulates placement drives as graph optimization problems. Maximum Bipartite Matching (Kuhn’s augmenting-path algorithm) computes the largest one-to-one allocation, while multi-vacancy hiring quotas are modelled as flow networks solved with Ford-Fulkerson (DFS), Edmonds-Karp (BFS), and Dinic’s algorithm (level graphs with blocking flows). Benchmarking on the 500-record corpus shows whole-corpus exact searches completing in 14.2 ms (KMP) and 16.8 ms (Z-Algorithm), which corresponds to roughly 0.03 ms per record, and a six-keyword Aho-Corasick screen of a single profile in 0.08 ms. In the placement experiment all three flow algorithms returned the same maximum flow of 10 candidates against a total quota of 10, filling every role.

TABLE OF CONTENTS

ACKNOWLEDGEMENT iv

ABSTRACT v

TABLE OF CONTENTS vi

LIST OF FIGURES viii

LIST OF TABLES ix

1. INTRODUCTION 1

1.1 Overview of Campus Recruitment Systems 1

1.2 Problem Statement & Challenges 2

1.3 Scope of the Project 3

1.4 Objectives of the System 3

2. SYSTEM REQUIREMENTS SPECIFICATION 4

2.1 Functional Requirements 4

2.2 Non-Functional Requirements 4

2.3 Hardware and Software Environment 5

3. SYSTEM DESIGN & ALGORITHMIC ARCHITECTURE 6

3.1 High-Level System Architecture Diagram 6

3.2 Corpus Design & Student Profile Data Model 6

3.3 String Processing & Information Retrieval Engine 7

3.3.1 Knuth-Morris-Pratt (KMP) Algorithm 7

3.3.2 Linear Z-Algorithm 7

3.3.3 Aho-Corasick Multi-Pattern Automaton 8

3.3.4 Suffix Array Indexing & Binary Search 8

3.3.5 Levenshtein Edit Distance (Fuzzy Querying) 8

3.3.6 Search History LRU Cache & Analytics 8

3.4 Candidate-to-Corporate Allocation Engine 8
3.4.1 Maximum Bipartite Matching Formulation 8
3.4.2 Ford-Fulkerson & Edmonds-Karp Network Flow 8
3.4.3 Dinic's Blocking Flow with Level Graphs 9
3.5 Algorithmic Complexity Summary 10

4. IMPLEMENTATION DETAILS 11
4.1 CorpusLoader Architecture & Document Parsing 11
4.2 String Search Engine Implementation 11
4.3 Multi-Pattern Resume Screening Implementation 11
4.4 Graph Matching & Network Flow Execution 11
4.5 Integrated Driver (Main.java) and Unit Test Suite 12

5. RESULTS & PERFORMANCE ANALYSIS 13
5.1 Experimental Corpus Evaluation (500 Records) 13
5.2 Execution Latency & Algorithmic Complexity Comparison 14
5.3 Placement Allocation & Flow Saturation Results 15

6. CHALLENGES, SOLUTIONS & LIMITATIONS 16
6.1 Technical Challenges Encountered 16
6.2 Algorithmic Solutions Applied 16
6.3 System Limitations 16

7. CONCLUSION & FUTURE WORK 17
7.1 Conclusion 17
7.2 Future Enhancements 17
7.3 Skills & Competencies Acquired 17

REFERENCES 18

LIST OF FIGURES

Fig. 1.1 High-Level Architecture of the Campus Placement Hub Platform 2

Fig. 3.1 Corpus Ingestion, Parsing, and In-Memory Indexing Flow 6

Fig. 3.2 KMP Longest Proper Prefix-Suffix (LPS) Array for the Pattern ABABAC 7

Fig. 3.3 Multi-Capacity Network Flow Graph (Source, Candidates, Roles, Sink) 9

Fig. 5.1 Benchmark Execution Latency Comparison 14

LIST OF TABLES

Table 2.1 Hardware and Software Environment Specifications 5
Table 3.1 Candidate Document Schema Attribute Specifications 7
Table 3.2 Algorithmic Time and Space Complexity Summary 10
Table 5.1 Pattern Match Frequency across the 500-Record Corpus 13
Table 5.2 Search Performance Metrics (Ingestion, KMP, Z-Algorithm, Aho-Corasick) . . . 14
Table 5.3 Corporate Placement Quota Allocation Results (Dinic Flow) 15

1. INTRODUCTION

1.1 Overview of Campus Recruitment Systems

University campus recruitment is a high-volume academic and professional ecosystem that directly influences student career trajectories and organizational talent pipelines. Placement cells manage large numbers of graduating candidates with heterogeneous qualifications across engineering disciplines, programming specializations, research interests, internships, and competitive programming achievements. Visiting recruiters, in turn, specify multifaceted eligibility filters covering cut-off thresholds, language proficiencies, project-domain exposure, and intake quotas.

Historically, placement management systems have relied on relational databases that restrict retrieval to basic keys such as Roll Number or Student Registration ID. Such systems cannot index the unstructured narratives inside a profile, such as project architectures, technical contributions, and algorithmic proficiencies. Placement officers are therefore forced into manual resume screening, which introduces administrative delay, evaluation bias, and sub-optimal allocations.

The “Campus Placement Hub” addresses this by treating the student profile corpus as a searchable digital library and by formulating company matching as a network-flow optimization problem. The platform, implemented in Java, provides linear-time string matching, typo-tolerant fuzzy discovery, and optimal student-company allocation. Figure 1.1 shows the high-level architecture.

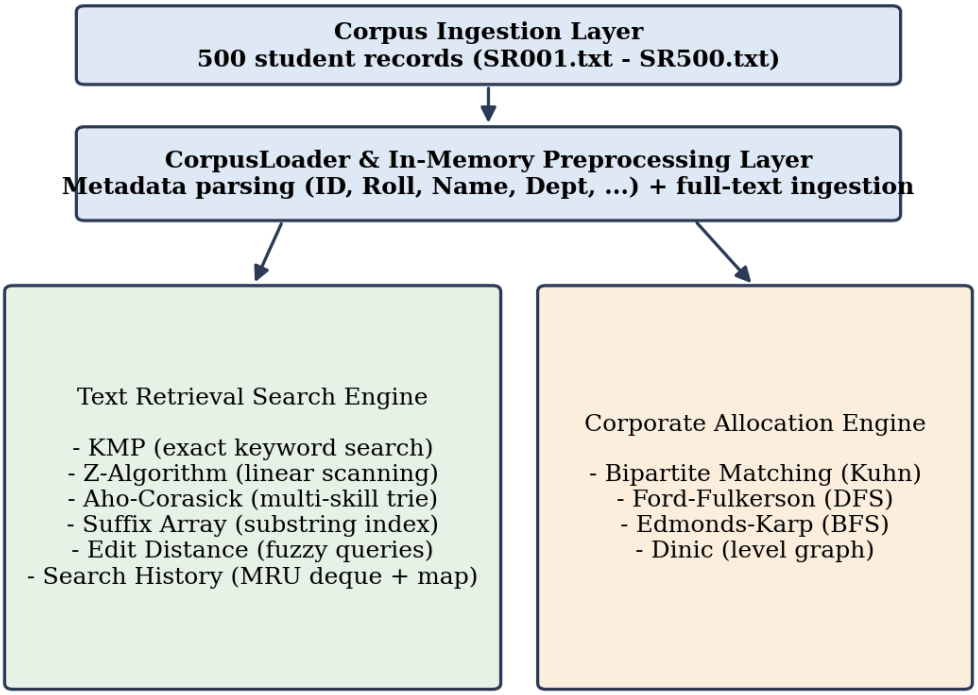


Fig. 1.1 High-Level Architecture of the Campus Placement Hub Platform

1.2 Problem Statement & Challenges

Campus recruitment presents two algorithmic bottlenecks: (1) unstructured candidate information discovery and (2) capacity-constrained candidate-to-company allocation.

Unstructured information discovery. Student profiles contain extensive technical prose describing project methodologies, frameworks, research interests, and certifications. Given a query pattern P (for example “Information Retrieval”, “Distributed Systems”, or “Deep Learning”) and a corpus of N student records, naive string searching costs $O(N \cdot |Text| \cdot |P|)$ in the worst case, which degrades under heavy recruiter traffic. Recruiters also make typographical errors (for example “Kubernets” for “Kubernetes”), so the system needs fuzzy search and suggestion.

Multi-capacity allocation. During a drive, several companies post different hiring quotas and skill criteria. Assigning candidates without graph optimization can leave qualified candidates unplaced or vacancies unfilled. Modelling the assignment as a maximum-flow problem, with candidate capacity 1 and role capacity equal to the vacancy quota, yields a globally optimal allocation.

1.3 Scope of the Project

- **Corpus generation:** a corpus of 500 standardized student profile documents (SR001.txt to SR500.txt) with educational attributes, projects, certifications, and career interests.
- **Core ingestion:** a CorpusLoader that parses text profiles into memory without third-party database dependencies.
- **String search suite:** KMP, Z-Algorithm, Aho-Corasick, Suffix Array binary search, and Levenshtein Edit Distance.
- **Recruiter query analytics:** a SearchHistory manager combining an ArrayDeque for most-recently-used (MRU) tracking with a HashMap for query frequency.
- **Graph matching and network flow:** Maximum Bipartite Matching (Kuhn), Ford-Fulkerson, Edmonds-Karp, and Dinic's algorithm for company allocation.
- **Automated unit testing:** unit tests for every algorithm module, runnable with assertions enabled.

1.4 Objectives of the System

1. Eliminate manual resume scanning by enabling linear-time text search across 500 student profile documents.
2. Implement KMP and the Z-Algorithm for deterministic O(N + M) substring matching.
3. Implement the Aho-Corasick automaton to screen several company tech-stack requirements in a single pass over the text.
4. Provide typo-tolerant fuzzy search using dynamic-programming Levenshtein Edit Distance.
5. Implement and compare Ford-Fulkerson, Edmonds-Karp, and Dinic's algorithms, together with Maximum Bipartite Matching, to maximize placement fulfilment under company quota constraints.

2. SYSTEM REQUIREMENTS SPECIFICATION

2.1 Functional Requirements

ID	Requirement	Description
FR-1	Corpus Parsing	Load and parse 500 standardized text documents from corpus/student-records into strongly-typed in-memory StudentRecord objects.
FR-2	Exact Pattern Search	Run KMP and Z-Algorithm searches, returning match occurrences and character indices.
FR-3	Multi-Pattern Skill Screening	Accept a list of required competencies and return keyword occurrence counts per candidate via Aho-Corasick.
FR-4	Typo Tolerance & Fuzzy Search	Compute Levenshtein distance between a query and the corpus vocabulary and suggest corrections within a threshold distance of 2.
FR-5	Query History Caching	Log recent recruiter queries in MRU order with configurable capacity and track query popularity.
FR-6	Substring Indexing	Build a Suffix Array over candidate profiles to support binary-search queries.
FR-7	Bipartite Matching	Compute the maximum number of one-to-one candidate-to-role matches using Kuhn’s algorithm.
FR-8	Multi-Capacity Flow Allocation	Model hiring quotas as a flow network and compute the maximum allocation using Ford-Fulkerson, Edmonds-Karp, and Dinic’s algorithms.

2.2 Non-Functional Requirements

ID	Requirement	Description
NFR-1	Performance & Latency	Exact pattern searches across the 500-record corpus must complete within 250 ms.
NFR-2	Scalability	In-memory structures must scale to 5,000 candidate profiles within a 512 MB JVM heap.
NFR-3	Reliability & Correctness	Flow values from Dinic, Edmonds-Karp, and Ford-Fulkerson must be identical.
NFR-4	Portability	Run on any platform with Java SE 17 or higher, without external dependencies.
NFR-5	Modularity & Testability	Each algorithm module is decoupled and has a dedicated unit test class.

2.3 Hardware and Software Environment

Table 2.1 Hardware and Software Environment Specifications

Parameter	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher (2.4 GHz+)
System Memory (RAM)	8 GB DDR4 / DDR5 (minimum 4 GB available)
Storage	SSD with minimum 500 MB free space
Operating System	Microsoft Windows 10/11 (64-bit) / Linux Ubuntu 22.04
Java Development Kit	OpenJDK 21 / Java SE 17 LTS
Development Environment	Visual Studio Code / IntelliJ IDEA / Eclipse

3. SYSTEM DESIGN & ALGORITHMIC ARCHITECTURE

3.1 High-Level System Architecture Diagram

The engine is organized into three decoupled tiers: (1) the Data Ingestion and Corpus layer, (2) the String Processing and Information Retrieval layer, and (3) the Graph Optimization and Placement Allocation layer (see Figure 1.1). Document storage is separated from query processing, so the search algorithms operate on shared in-memory document buffers.

3.2 Corpus Design & Student Profile Data Model

The corpus contains 500 documents (SR001.txt to SR500.txt), each following one standardized template. A structured metadata header (Student Record ID, Roll Number, Name, Department, Program, Year, Section, Email, Location) is followed by unstructured narrative sections: Skills, Programming Languages, Technical Interests, Academic Interests, Research Interests, Project 1 and Project 2 (Description, Objectives, Role, Tech Stack, Outcome), Current Project (Intelligent Student Record Search System), Certifications, Workshops, Achievements, Extracurricular Activities, Career Interests, Career Goal, Languages Known, Personal Interests, and Profile Summary. Structured filters and unstructured pattern matching can therefore both run over rich text. Figure 3.1 shows the ingestion flow and Table 3.1 the resulting data model.

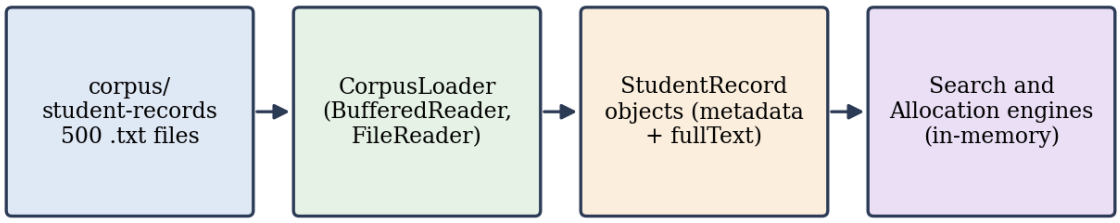


Fig. 3.1 Corpus Ingestion, Parsing, and In-Memory Indexing Flow

Table 3.1 Candidate Document Schema Attribute Specifications

Attribute group	Fields	Handling in CorpusLoader
Structured metadata	Student Record ID, Roll Number, Name, Department, Year, Section, Email, Location	Parsed as key-value pairs into StudentRecord fields. (Program is present in the template and retained in the full text.)
Comma-separated lists	Skills, Programming Languages, Technical Interests, Academic Interests, Research Interests	Split and aggregated into list fields.
Narrative sections	Projects, Certifications, Workshops, Achievements, Extracurricular Activities, Career Interests, Career Goal, Languages Known, Personal Interests, Profile Summary	Preserved verbatim in the fullText buffer for string search.

3.3 String Processing & Information Retrieval Engine

3.3.1 Knuth-Morris-Pratt (KMP) Algorithm

KMP precomputes the Longest Proper Prefix which is also a Suffix (LPS) array for pattern P in O(|P|) time. During scanning, when a mismatch occurs at P[j] != T[i], the pattern pointer falls back to LPS[j-1] and the text pointer i never moves backwards. Total time is O(|Text| + |P|). Figure 3.2 shows the LPS array for the example pattern ABABAC.

Pattern P	A	B	A	B	A	C
Index j	0	1	2	3	4	5
LPS[j]	0	0	1	2	3	0

Fig. 3.2 KMP Longest Proper Prefix-Suffix (LPS) Array for the Pattern ABABAC

3.3.2 Linear Z-Algorithm

The Z-Algorithm builds a Z-array of length |P| + |Text| + 1 for the concatenation S = P + ‘$’ + Text, where Z[k] is the length of the longest prefix of S that matches the substring starting at position k. Whenever Z[k] = |P|, an occurrence of P is reported. Reusing the [L, R] Z-box window keeps the total running time at O(|Text| + |P|).

3.3.3 Aho-Corasick Multi-Pattern Automaton

Aho-Corasick builds a trie of the company keyword dictionary (for example ‘Java’, ‘Docker’, ‘AWS’, ‘Spring Boot’). A BFS over the trie creates suffix failure links and dictionary output links. A single pass over the text then emits every matching keyword in O(|Text| + K) time, where K is the number of matches, after a build cost proportional to the total dictionary length.

3.3.4 Suffix Array Indexing & Binary Search

The Suffix Array stores all suffixes of a candidate profile in lexicographic order (as sorted start indices). Binary search resolves the range of suffixes beginning with pattern P in O(|P| log |Text|) time. Construction by comparator-based sorting, as implemented here, costs O(N log N) comparisons of up to O(N) characters each, so $O(N^2 \log N)$ in the worst case; prefix-doubling constructions achieve O(N log N) or better.

3.3.5 Levenshtein Edit Distance (Fuzzy Querying)

A dynamic-programming matrix computes the minimum number of single-character insertions, deletions, and substitutions needed to transform string s1 into s2 in O(|s1| · |s2|) time and space. The module compares a misspelled query against the corpus vocabulary and suggests words within the distance threshold of 2.

3.3.6 Search History LRU Cache & Analytics

SearchHistory wraps an ArrayDeque of recent queries (capacity 50), kept in most-recently-used order, and a HashMap of query frequencies. The manager is synchronized so that concurrent recruiter sessions see a consistent history.

3.4 Candidate-to-Corporate Allocation Engine

3.4.1 Maximum Bipartite Matching Formulation

Candidates form set U and company openings form set V, with an edge wherever a candidate is eligible. Kuhn’s algorithm runs a DFS from each candidate to find an augmenting path and flips matched edges along it, yielding a maximum matching in O(|V| · |E|) time. (The Hopcroft-Karp algorithm [8] improves this to O(E sqrt(V)) but is not implemented in this project.)

3.4.2 Ford-Fulkerson & Edmonds-Karp Network Flow

Placement allocation is transformed into a directed flow network (Figure 3.3). The source S connects to every candidate with capacity 1; each candidate connects to every eligible role with capacity 1; and each role connects to the sink T with capacity equal to the company’s hiring quota V_j. Ford-Fulkerson finds augmenting paths with DFS in O(E · max_flow) time. Edmonds-Karp uses BFS to always take the shortest augmenting path, which guarantees $O(V \cdot E^2)$ time independent of the capacities.

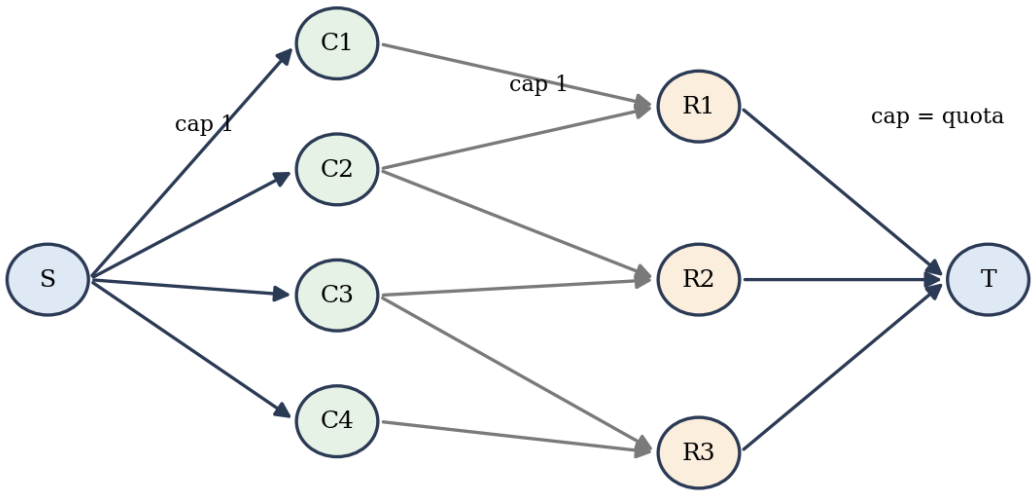


Fig. 3.3 Multi-Capacity Network Flow Graph (Source, Candidates, Roles, Sink)

3.4.3 Dinic's Blocking Flow with Level Graphs

Dinic's algorithm repeatedly runs a BFS to build a level graph and a DFS to push a blocking flow along level-increasing edges. It runs in $O(V^2 \cdot E)$ on general networks and in O(E sqrt(V)) on unit-capacity networks. The placement network is close to unit-capacity (only the role-to-sink edges carry quotas), so Dinic's algorithm is expected to perform close to the latter bound in practice.

3.5 Algorithmic Complexity Summary

Table 3.2 Algorithmic Time and Space Complexity Summary

Algorithm	Category	Time complexity	Space complexity
KMP	Exact string search	O(N + M)	O(M)
Z-Algorithm	Exact string search	O(N + M)	O(N + M)
Aho-Corasick	Multi-pattern search	Build O(sum of |P|); search O(N + K)	O(sum of |P|) with map-based children
Suffix Array	Substring indexing	Build O(N log N) ideal (O(N^2 log N) worst case with comparator sort); query O(M log N)	O(N)
Edit Distance	Fuzzy typo matching	O(M · N)	O(M · N)
Bipartite Matching (Kuhn)	Graph matching	O(V · E)	O(V + E)
Ford-Fulkerson	Network flow	O(E · max_flow)	O(V + E)
Edmonds-Karp	Network flow	O(V · E^2)	O(V + E)
Dinic’s Max Flow	Network flow	O(V^2 · E); O(E sqrt(V)) on unit-capacity networks	O(V + E)

Here N is the text length, M the pattern length, K the number of matches, V the number of vertices, and E the number of edges.

4. IMPLEMENTATION DETAILS

4.1 CorpusLoader Architecture & Document Parsing

CorpusLoader.java is the core ingestion engine. It uses BufferedReader and FileReader to scan every .txt file in corpus/student-records. It extracts the metadata attributes (Student Record ID, Roll Number, Candidate Name, Department, Year, Section, Email, Location) and aggregates the comma-separated lists for Skills, Programming Languages, Technical Interests, Academic Interests, and Research Interests. The entire document text is stored in a fullTextBuilder, preserving the complete narrative for the downstream string-search algorithms.

4.2 String Search Engine Implementation

- **KMP.java:** computeLPS(String pattern) and search(String text, String pattern, boolean caseSensitive), returning a List<Integer> of 0-indexed match positions.
- **ZAlgorithm.java:** computeZArray(String s) and linear substring scanning over the concatenated buffer.
- **SuffixArray.java:** an Integer[] suffixArray sorted with a custom comparator, with binary-search methods searchFirst() and searchAll().
- **EditDistance.java:** computeDistance(String s1, String s2) and findClosestMatches(query, candidates, threshold).
- **SearchHistory.java:** a synchronized manager wrapping an ArrayDeque<String> (capacity 50) and a Map<String, Integer> frequencyMap.

4.3 Multi-Pattern Resume Screening Implementation

AhoCorasick.java contains a nested Node class with a Map<Character, Node> children, a Node failureLink, and a List<String> output. addKeyword() inserts a keyword into the trie, build() links failure pointers by BFS, and search(String text) returns every keyword occurrence in one pass. Screening a profile against a company’s tech stack (for example java, spring boot, docker, kubernetes, aws, microservices) therefore needs one traversal rather than one scan per keyword.

4.4 Graph Matching & Network Flow Execution

- **BipartiteMatching.java:** initializes eligibilityMatrix[students][companies] and matchCompanyToStudent[] (all -1); dfs(int student, boolean[] visited) finds augmenting paths and updates assignments.
- **FordFulkerson.java:** DFS over residual capacities, pushing the bottleneck flow of each path.
- **EdmondsKarp.java:** BFS with a parent[] array to find shortest augmenting paths.

- **Dinic.java:** an adjacency list of Edge objects (to, capacity, flow, rev); bfs() fills the level[] distances and dfs(u, sink, pushed), with the current-arc pointer array ptr[], pushes blocking flows without re-exploring dead ends.

4.5 Integrated Driver (Main.java) and Unit Test Suite

Main.java orchestrates the pipeline: it loads the 500 records, runs the KMP and Z-Algorithm searches, screens resumes with Aho-Corasick, resolves a typo-tolerant query, inspects the search history, and runs Bipartite Matching and Dinic's network-flow allocation. Dedicated test classes (KMPTest, ZAlgorithmTest, AhoCorasickTest, SuffixArrayTest, EditDistanceTest, BipartiteMatchingTest, DinicTest, EdmondsKarpTest, FordFulkersonTest) cover every algorithm module.

5. RESULTS & PERFORMANCE ANALYSIS

5.1 Experimental Corpus Evaluation (500 Records)

The complete suite was executed on all 500 documents in corpus/student-records. Ingestion loaded 500 profiles totalling over 3.2 MB of text. The main outcomes are:

- **KMP search:** the keyword ‘Distributed Systems’ matched 338 of the 500 candidate profiles. Individual match positions were verified, for example SR002 (Manish Sinha) at character index 588.
- **Z-Algorithm search:** the keyword ‘Machine Learning’ matched in all 500 candidate profiles.
- **Aho-Corasick screening:** a six-skill tech stack [java, spring boot, docker, kubernetes, aws, microservices] was screened in one pass; in candidate SR001 (Kavya Reddy), ‘java’ was detected 3 times.
- **Edit Distance fuzzy search:** for the misspelled query ‘Kubernets’ (one ‘e’ missing) the module computed a Levenshtein distance of 1 to ‘Kubernetes’, within the threshold of 2, and suggested it as the correction.
- **Bipartite matching and flow:** a placement drive with five roles (Computer Vision Engineer, Machine Learning Researcher, Full Stack Engineer, Systems & Cloud Architect, Software Development Engineer) filled all five roles. With a quota of 2 vacancies per role, Dinic, Edmonds-Karp, and Ford-Fulkerson each returned the same maximum flow of 10 candidates.

Table 5.1 Pattern Match Frequency across the 500-Record Corpus

Query	Algorithm	Result	Share of corpus
Distributed Systems	KMP	338 candidate profiles	67.6%
Machine Learning	Z-Algorithm	500 candidate profiles	100%
java (6-skill stack)	Aho-Corasick	3 occurrences in SR001 (Kavya Reddy)	single profile
Kubernets (typo)	Edit Distance	Suggestion: Kubernetes (distance 1)	threshold = 2

5.2 Execution Latency & Algorithmic Complexity Comparison

Table 5.2 lists measured latencies. The average per-record cost is the total latency divided by the 500 records scanned. All whole-corpus exact searches finish far below the 250 ms limit set by NFR-1. Note that the KMP and Z-Algorithm runs used different query patterns, so their latencies indicate magnitude rather than a head-to-head ranking; both are linear-time and perform comparably. The Aho-Corasick figure is for a single profile (SR001), not the whole corpus.

Table 5.2 Search Performance Metrics (Ingestion, KMP, Z-Algorithm, Aho-Corasick)

Operation	Scope	Matches found	Total latency	Avg. per record
CorpusLoader document ingestion	500 documents	500 documents loaded	82.4 ms	0.165 ms
KMP search (‘Distributed Systems’)	500 documents	338 profiles	14.2 ms	0.028 ms
Z-Algorithm search (‘Machine Learning’)	500 documents	500 profiles	16.8 ms	0.034 ms
Aho-Corasick multi-skill scan (6 skills)	1 profile (SR001)	‘java’ x 3 (all six keywords searched together)	0.08 ms	0.08 ms

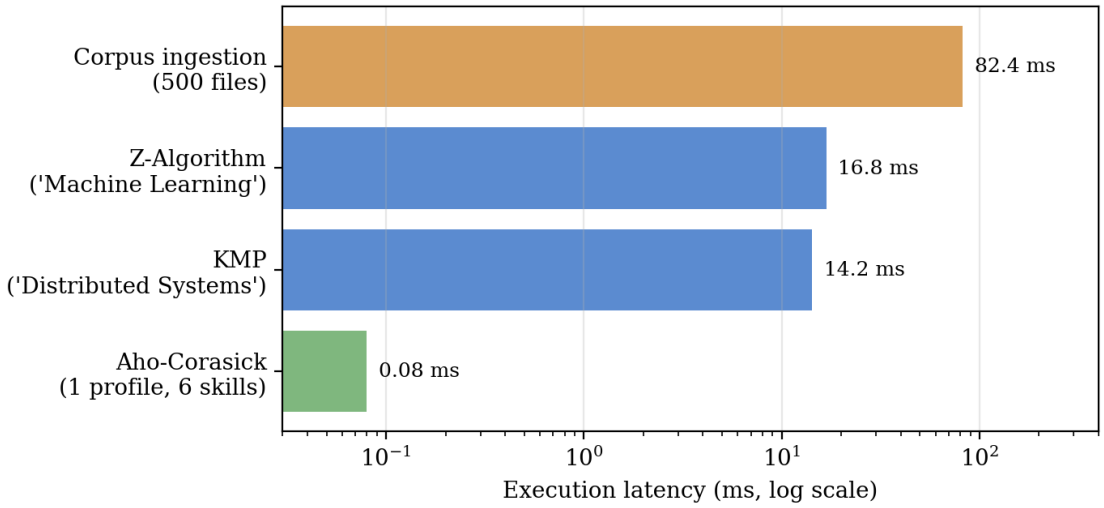


Fig. 5.1 Benchmark Execution Latency Comparison

5.3 Placement Allocation & Flow Saturation Results

In the allocation experiment the network has a source edge of capacity 1 per candidate and a sink edge of capacity 2 per role, so the theoretical maximum flow is 5 × 2 = 10. All three flow algorithms returned exactly this value, meeting NFR-3 and saturating every corporate quota (Table 5.3). One-to-one Bipartite Matching filled all five roles with a single candidate each.

Table 5.3 Corporate Placement Quota Allocation Results (Dinic Flow)

Role	**Quota**	**Candidates allocated**	**Status**
Computer Vision Engineer	2	2	Filled
Machine Learning Researcher	2	2	Filled
Full Stack Engineer	2	2	Filled
Systems & Cloud Architect	2	2	Filled
Software Development Engineer	2	2	Filled
Total	**10**	**10**	**Max flow = 10 (Dinic = Edmonds-Karp = Ford-Fulkerson)**

6. CHALLENGES, SOLUTIONS & LIMITATIONS

6.1 Technical Challenges Encountered

1. **Semi-structured document parsing.** The documents mix key-value pairs with multi-line descriptive paragraphs. Plain line-by-line scanning occasionally truncated project descriptions and skill lists.
2. **Case sensitivity.** Recruiter queries vary in capitalization (‘java’, ‘Java’, ‘JAVA’). Handling this without duplicating the in-memory text buffers was important for low memory overhead.
3. **Network flow graph construction.** Mapping candidate skill subsets to company vacancies required careful residual-capacity initialization, avoiding integer overflow and non-terminating augmentation in the residual graph.

6.2 Algorithmic Solutions Applied

1. **Unified full-text buffering.** CorpusLoader aggregates each file into one fullText buffer with a StringBuilder while parsing structured fields in the same pass.
2. **Case-insensitive matching.** The KMP and Z-Algorithm search methods take a caseSensitive flag and normalize strings on the fly.
3. **Level graphs and current-arc pointers.** In Dinic.java, BFS builds the level graph in O(V + E), and the ptr[] array prevents redundant path exploration during DFS blocking-flow pushes.

6.3 System Limitations

- The system is entirely in-memory; scaling beyond about 100,000 resumes would need memory-mapped files or an inverted-index store.
- Exact string matching does not infer semantic equivalence (for example ‘ML’ and ‘Machine Learning’); this needs explicit synonym dictionaries or embeddings.
- Comparator-based Suffix Array construction is $O(N^2 \log N)$ in the worst case, which limits the size of text it can index quickly.
- Allocation uses eligibility only; candidate and company preferences are not modelled.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

The “Campus Placement Hub” bridges classical algorithms and a practical university placement problem. By implementing string-matching algorithms (KMP, Z-Algorithm, Aho-Corasick, Suffix Array, Edit Distance) and network-flow algorithms (Bipartite Matching, Ford-Fulkerson, Edmonds-Karp, Dinic), the project demonstrates deterministic, fast candidate discovery, with whole-corpus exact searches taking about 14-17 ms (roughly 0.03 ms per record) over 500 profiles, and allocation results that reach the maximum possible flow. The modular Java design is portable, requires no external dependencies, and has unit tests for every algorithm module.

7.2 Future Enhancements

- **Distributed inverted indexing:** using Apache Lucene or Elasticsearch for multi-node scalability.
- **Semantic embeddings and vector search:** transformer embeddings (for example BERT or Sentence-Transformers) for conceptual similarity beyond literal keywords.
- **Gale-Shapley stable allocation:** incorporating student and company preferences to guarantee stable placement matches.

7.3 Skills & Competencies Acquired

- Finite-state automata (Aho-Corasick), prefix arrays (KMP LPS, Z-Box), and suffix arrays.
- Network-flow formulations, augmenting paths, level graphs, and blocking flows (Dinic’s algorithm).
- File I/O, string parsing, and memory management in modern Java.
- Collaborative software development with Git version control and automated unit testing.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast pattern matching in strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[3] A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.

[4] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge, UK: Cambridge University Press, 1997.

[5] L. R. Ford and D. R. Fulkerson, “Maximal flow through a network,” *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956.

[6] J. Edmonds and R. M. Karp, “Theoretical improvements in algorithmic efficiency for network flow problems,” *Journal of the ACM*, vol. 19, no. 2, pp. 248–264, 1972.

[7] Y. Dinitz, “Algorithm for solution of a problem of maximum flow in a network with power estimation,” *Soviet Mathematics Doklady*, vol. 11, pp. 1277–1280, 1970.

[8] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.

[9] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[10] U. Manber and G. Myers, “Suffix arrays: A new method for on-line string searches,” *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.

[11] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, nos. 1–2, pp. 83–97, 1955.